

Approximative Darstellung reeller Zahlen

Max Recker

`max.recker@uni-bielefeld.de`

5.5.2026

Übersicht

1. 4.1 b-adische Darstellung reeller Zahlen
2. 4.2 Maschinenzahlen
3. 4.3 Rundung
4. 4.4 Maschinenzahlarithmetik

Ziel des Kapitels

- ▶ Jede reelle Zahl soll in einer Basis $b \geq 2$ dargestellt werden.

Bekannte Basen:

$$b \in \mathbb{N}, b \geq 2$$

$b = 10$: Dezimalsystem

$b = 2$: Binärsystem

$b = 8$: Oktalsystem

$b = 16$: Hexadezimalsystem

Diese Darstellung ist die Grundlage für spätere Maschinenzahlen (4.2).

Satz 4.1: b-adische Darstellung reeller Zahlen

Satz

Sei $b \in \mathbb{N}$ mit $b \geq 2$. Sei $x \in \mathbb{R} \setminus \{0\}$. Dann existieren eindeutig bestimmte Zahlen

$$E \in \mathbb{Z}, \quad \sigma \in \{-1, 1\}, \quad z_i \in \{0, 1, \dots, b-1\}$$

für $i \in \mathbb{N} \cup \{0\}$ mit

$$\{i \in \mathbb{N} \mid z_i \neq b-1\} \text{ unendlich}, \quad z_0 \neq 0$$

und

$$x = \sigma \cdot b^E \cdot \left(\sum_{i=0}^{\infty} z_i b^{-i} \right).$$

Dies heißt die (normalisierte) b-adische Darstellung von x .

Bestandteile von x

$$x = \sigma \cdot b^E \cdot \left(\sum_{i=0}^{\infty} z_i b^{-i} \right)$$

- ▶ x : die Zahl, die dargestellt werden soll
- ▶ $\sigma \in \{-1, 1\}$: Vorzeichen
- ▶ b : Basis des Zahlensystems
- ▶ $E \in \mathbb{Z}$: Exponent / Größenordnung
- ▶ z_i : i -te Ziffer der Zahl
- ▶ b^{-i} : Stellenwert der jeweiligen Ziffer

Beispiel:

$$3,1415 \Rightarrow z_0 = 3, z_1 = 1, z_2 = 4, z_3 = 1, z_4 = 5$$

$$\Rightarrow \text{Zahl} = \text{Vorzeichen} \cdot \text{Größe} \cdot \text{Mantisse}$$

Behauptung und Beginn des Beweises

Behauptung: Es gilt

$$1 \leq \sum_{i=0}^{\infty} z_i b^{-i} < \sum_{i=0}^{\infty} (b-1)b^{-i} = b$$

d.h. der Grenzwert existiert, weil die Folge monoton wachsend ist und durch b nach oben beschränkt.

Bew. der Aussage:

1. Sei $x \in \mathbb{R} \setminus \{0\}$.

Dann gilt offensichtlich:

$$\sigma = \begin{cases} -1, & x < 0 \\ 1, & x > 0 \end{cases}$$

Für

$$z_i \in \{0, 1, \dots, b-1\}, \quad i \in \mathbb{N} \cup \{0\}$$

mit

$$\{i \in \mathbb{N} \mid z_i \neq b-1\} \neq \emptyset \quad \text{und} \quad z_0 \neq 0$$

gilt.

Warum gilt $1 \leq \sum z_i b^{-i}$?

(2) Warum ist

$$1 \leq \sum_{i=0}^{\infty} z_i b^{-i} ?$$

Da $z_0 \neq 0$ und $z_0 \in \{0, 1, \dots, b-1\}$, folgt:

$$z_0 \geq 1.$$

Damit gilt:

$$\sum_{i=0}^{\infty} z_i b^{-i} = z_0 + \sum_{i=1}^{\infty} z_i b^{-i} \geq z_0 \geq 1.$$

Gleichheit genau dann, wenn

$$\forall i \in \mathbb{N} : z_i = 0.$$

Dann:

$$\sum_{i=0}^{\infty} z_i b^{-i} = 1.$$

Sonst immer größer als 1.

Warum ist $\sum (b-1)b^{-i} = b$?

(3) Warum ist

$$\sum_{i=0}^{\infty} (b-1)b^{-i} = b ?$$

$$\sum_{i=0}^{\infty} (b-1)b^{-i} = (b-1) \sum_{i=0}^{\infty} b^{-i}$$

Dies ist eine geometrische Reihe mit Quotient

$$q = \frac{1}{b}.$$

Daher:

$$\sum_{i=0}^{\infty} b^{-i} = \frac{1}{1 - \frac{1}{b}} = \frac{b}{b-1}.$$

Einsetzen ergibt:

$$(b-1) \cdot \frac{b}{b-1} = b.$$

Ziel der Normierung

Behauptung:

$$E := \lfloor \log_b |x| \rfloor$$

Wir wollen x so schreiben, dass

$$x = \sigma \cdot b^E \cdot a_0 \quad \text{mit} \quad 1 \leq a_0 < b.$$

Dann folgt:

$$a_0 = b^{-E} |x|.$$

Gesucht ist also ein Exponent E , für den gilt:

$$1 \leq b^{-E} |x| < b.$$

Umformen liefert:

$$1 \leq \frac{|x|}{b^E} < b \quad \Longleftrightarrow \quad b^E \leq |x| < b^{E+1}.$$

Bestimmung des Exponenten E

Aus

$$b^E \leq |x| < b^{E+1}$$

wenden wir den Logarithmus zur Basis b an:

$$\log_b(b^E) \leq \log_b(|x|) < \log_b(b^{E+1}).$$

Mit

$$\log_b(b^E) = E, \quad \log_b(b^{E+1}) = E + 1$$

folgt:

$$E \leq \log_b(|x|) < E + 1.$$

Damit muss gelten:

$$E = \lfloor \log_b(|x|) \rfloor.$$

Darstellung von a_0 mit Restglied

Behauptung: Für alle $n \in \mathbb{N} \cup \{0\}$ gilt

$$a_0 = \sum_{i=0}^n z_i b^{-i} + a_{n+1} b^{-n-1}.$$

Dabei ist:

- ▶ $\sum_{i=0}^n z_i b^{-i}$: bekannte Ziffern / Partialsumme
- ▶ $a_{n+1} b^{-n-1}$: Restglied

Der verbleibende Rest wird ab dem Index $n + 1$ zusammengefasst.

Ziel:

$$a_0 = \sum_{i=0}^{\infty} z_i b^{-i}.$$

Das Restglied verschwindet für $n \rightarrow \infty$.

Wichtig:

Falls

$$1 \leq x < b,$$

dann gilt bereits

$$a_0 = x,$$

also ist keine zusätzliche Normierung nötig.

Beweis per Induktion: Induktionsanfang

Induktionsanfang: $n = 0$

Zu zeigen:

$$a_0 = z_0 b^0 + a_1 b^{-1}.$$

$$a_0 = z_0 + \frac{a_1}{b}.$$

Wir wissen:

$$a_1 = b(a_0 - z_0).$$

Einsetzen liefert:

$$a_0 = z_0 + \frac{b(a_0 - z_0)}{b}.$$

$$a_0 = z_0 + (a_0 - z_0) = a_0.$$



Beweis per Induktion: Induktionsvoraussetzung

Induktionsvoraussetzung ($n - 1 \rightarrow n$):

Zu zeigen:

$$a_0 = \sum_{i=0}^n z_i b^{-i} + a_{n+1} b^{-n-1}.$$

Also lautet die Induktionsannahme:

$$a_0 = \sum_{i=0}^{n-1} z_i b^{-i} + a_n b^{-n}.$$

I.S. (Rekursionsformel)

Aus

$$a_{n+1} = b(a_n - z_n)$$

folgt

$$a_n = z_n + \frac{a_{n+1}}{b}.$$

Dann:

$$a_0 = \sum_{i=0}^{n-1} z_i b^{-i} + \left(z_n + \frac{a_{n+1}}{b} \right) b^{-n}$$

$$\Leftrightarrow a_0 = \sum_{i=0}^{n-1} z_i b^{-i} + z_n b^{-n} + a_{n+1} b^{-n-1}$$

$$\Leftrightarrow a_0 = \sum_{i=0}^n z_i b^{-i} + a_{n+1} b^{-n-1}$$



Übergang zum Grenzwert

Aus

$$x = \sigma \cdot b^E \cdot a_0$$

und

$$a_0 = \sum_{i=0}^n z_i b^{-i} + a_{n+1} b^{-n-1}$$

für jedes $n \in \mathbb{N} \cup \{0\}$ folgt:

$$x = \sigma \cdot b^E \cdot \lim_{n \rightarrow \infty} \left(\sum_{i=0}^n z_i b^{-i} \right),$$

sofern das Restglied verschwindet.

Also müssen wir zeigen:

$$a_{n+1} b^{-n-1} \rightarrow 0 \quad (n \rightarrow \infty).$$

Per Induktion gilt die Darstellung bereits für jedes endliche n . Nun geht man zum Grenzwert über.

Verschwinden des Restglieds

Zu zeigen:

$$a_{n+1}b^{-n-1} \rightarrow 0.$$

Dann bleibt:

$$a_0 = \lim_{n \rightarrow \infty} \sum_{i=0}^n z_i b^{-i} = \sum_{i=0}^{\infty} z_i b^{-i}.$$

Es wurde gezeigt:

$$1 \leq a_{n+1} < b.$$

Daraus folgt:

$$0 \leq a_{n+1}b^{-n-1} < b \cdot b^{-n-1} = b^{-n}.$$

Da $b^{-n} \rightarrow 0$, gilt mit dem Sandwichprinzip:

$$a_{n+1}b^{-n-1} \rightarrow 0.$$

Eindeutigkeit der normalisierten Darstellung

Annahme:

$$\{ i \in \mathbb{N} \mid z_i \neq b - 1 \}$$

sei endlich.

Dann existiert ein $n_0 \in \mathbb{N}$ mit

$$z_i = b - 1 \quad \text{für alle } i > n_0.$$

Behauptung:

Für jedes $x \in \mathbb{R} \setminus \{0\}$ gibt es genau eine normalisierte Darstellung

$$x = \sigma \cdot b^E \cdot \sum_{i=0}^{\infty} z_i b^{-i},$$

mit

$$\sigma \in \{-1, 1\}, \quad E \in \mathbb{Z}, \quad z_0 \neq 0.$$

Die Bedingung verhindert doppelte Darstellungen mit endlosem $b - 1$ -Schwanz.

1. Existenz der Darstellung

Existenz:

Zu jeder Zahl x lassen sich passende Ziffern z_i wählen, sodass

$$x = \sigma \cdot b^E \cdot \sum_{i=0}^{\infty} z_i b^{-i}$$

gilt.

Idee des Beweises:

- ▶ Die Zahl wird schrittweise in Basis b zerlegt.
- ▶ Für jede Stelle wird eine passende Ziffer z_i bestimmt.
- ▶ Dadurch entsteht die unendliche Reihe

$$\sum_{i=0}^{\infty} z_i b^{-i}.$$

- ▶ Würde ab einem gewissen Punkt nur noch $b - 1$ auftreten, entstünde eine alternative Darstellung.
- ▶ Deshalb wird dieser Fall ausgeschlossen.

2. Eindeutigkeit der Darstellung

Beweisidee zur Eindeutigkeit:

Angenommen, es gäbe zwei verschiedene normalisierte Darstellungen derselben Zahl.

Dann sei n die erste Stelle, an der sich die Ziffern unterscheiden.

- ▶ Vor der Stelle n sind beide Darstellungen identisch.
- ▶ An der Stelle n ist eine Darstellung größer als die andere.
- ▶ Dieser Unterschied besitzt den Stellenwert b^{-n} .
- ▶ Die späteren Stellen haben nur kleinere Gewichte ($b^{-n-1}, b^{-n-2}, \dots$) und können den Unterschied nicht mehr vollständig ausgleichen.

Daher können beide Summen nicht gleich sein.

⇒ Die Darstellung ist eindeutig.

Merksatz und Beispiele

Merksatz:

Wenn an der ersten unterschiedlichen Stelle eine Ziffer kleiner ist, dann kann der Rest diesen Unterschied nie vollständig aufholen.

Beispiele für $b = 2$:

$$\frac{1}{3} = 2^{-2} (2^0 + 2^{-2} + 2^{-4} + 2^{-6} + \dots) = (0, \overline{01})_2$$

$$8,1 = 2^3 (2^0 + 2^{-7} + 2^{-8} + 2^{-11} + 2^{-12} + \dots) = (1000, \overline{00011})_2$$

4.2 Maschinenzahlen

Motivation / Zusammenhang:

Da in einem Rechner nur endlich viele Stellen einer reellen Zahl dargestellt werden können,
nutzt man in Anlehnung an Satz 4.1 (normalisierte b -adische Darstellung von x)
die folgende Darstellung.

Definition 4.2.1: Maschinenzahl

Seien $b, m \in \mathbb{N}$ mit $b \geq 2$.

Eine b -adische m -stellige normalisierte Gleitkommazahl hat die Form

$$x = \sigma \cdot b^E \cdot \left(\sum_{i=0}^{m-1} z_i b^{-i} \right)$$

mit

$$\sigma \in \{-1, 1\}, \quad E \in \mathbb{Z},$$

$$z_0 \neq 0, \quad z_i \in \{0, \dots, b-1\} \quad \text{für alle } i \in \mathbb{N} \cup \{0\}.$$

$\Rightarrow$ Der einzige Unterschied zu Satz 4.1 ist:

Die Reihe läuft jetzt nur noch bis $m-1$, da wir eine m -stellige Zahl haben.

Warum nicht bis m ?

Weil wir bei 0 anfangen:

$$i = 0 \text{ (1. Stelle), } i = 1 \text{ (2. Stelle), } \dots, i = m-1 \text{ (m-te Stelle).}$$

Beispiel

1) Für $b = 10$ hat die Zahl 136,5 die normalisierte 5-stellige Darstellung:

$$136,5 = 1,3650 \cdot 10^2$$

In C++-Notation schreibt sich das als:

$$1.365e + 2$$

Maschinenzahlbereich

Für die Darstellung im Rechner sind geeignete Werte für b , m und den zulässigen Bereich

$$\{E_{\min}, \dots, E_{\max}\}$$

für E zu wählen.

Dies bestimmt den Maschinenzahlbereich:

$$F(b, m, E_{\min}, E_{\max})$$

Bedeutung:

- ▶ b : Basis des Zahlensystems
- ▶ m : Anzahl der Mantissenstellen
- ▶ $E_{\min}, E_{\max}$: erlaubter Exponentenbereich

In dieser Menge liegen Zahlen der Form:

$$x = \sigma \cdot b^E \cdot \left(\sum_{i=0}^{m-1} z_i b^{-i} \right),$$

Definition des Maschinenzahlbereichs

Der Maschinenzahlbereich ist die Menge der b -adischen, m -stelligen, normalisierten Gleitkommazahlen mit Exponent

$$E \in \{E_{\min}, \dots, E_{\max}\},$$

zuzüglich der Zahl 0.

Hinweis:

Die 0 muss extra hinzugefügt werden, da sie wegen der Normalisierung ($z_0 \neq 0$) nicht automatisch in der Menge enthalten ist.

Die Elemente eines (festen) Maschinenzahlbereichs heißen **Maschinenzahlen**.

Beispiel für Maschinenzahlen

Beispiel:

$$b = 10, \quad m = 3, \quad E \in \{-1, 0, 1\}$$

Dann sind mögliche Maschinenzahlen z. B.:

$$1,23 \cdot 10^1 = 12,3$$

$$9,99 \cdot 10^{-1} = 0,999$$

$$-4,56 \cdot 10^0 = -4,56$$

Und immer auch:

0

Anwendung in der Praxis

Im 1985 verabschiedeten IEEE-Standard 754 wird beispielsweise ein Datentyp `double` definiert mit

$$b = 2, \quad m = 53, \quad E \in \{-1022, \dots, 1023\}.$$

Dabei steht IEEE für:

The Institute of Electrical and Electronics Engineers

Wir schreiben:

$$F_{\text{double}} = F(2, 53, -1022, 1023).$$

Zur Darstellung dieser Zahlen reichen 64 Bits aus:

- ▶ 1 Bit für das Vorzeichen
- ▶ 52 Bits für die Mantisse
- ▶ 11 Bits für den Exponenten

IEEE 754: Hidden Bit

Wir schreiben:

$$F_{\text{double}} = F(2, 53, -1022, 1023).$$

Zur Darstellung dieser Zahlen reichen 64 Bits aus:

- ▶ 1 Bit für das Vorzeichen
- ▶ 52 Bits $z_1, \dots, z_{52}$ für die Mantisse
- ▶ 11 Bits für den Exponenten

Da wegen der Normalisierung das nullte Bit z_0 immer den Wert 1 hat, kann dieses weggelassen werden.

Es wird daher als **hidden bit** bezeichnet.

Deshalb genügen 52 Mantissenbits bei $m = 53$.

IEEE 754: Bias-Darstellung

Für den Exponenten wird die sogenannte **Bias-Darstellung** benutzt:
Eine Konstante (Bias) wird hinzuaddiert, um alle Exponenten positiv zu speichern.

Im konkreten Fall `double` wird

1023

hinzuaddiert.

Beispiele:

$$E = 0 \rightarrow 1023$$

$$E = 5 \rightarrow 1028$$

$$E = -3 \rightarrow 1020$$

Für den C++-Datentyp `double` schreibt der Standard nicht exakt vor, wie dies intern umgesetzt wird

Bit-Aufbau der Gleitkommadarstellung

Die Abspeicherung in Bits für die normalisierte Gleitkommadarstellung sieht wie folgt aus:

1 Bit	11 Bits	52 Bits
$\frac{1 + \sigma}{2}$	$E + 1023$	$z_1, z_2, \dots, z_{52}$

Bedeutung:

- ▶ Vorzeichenbit:

$$\sigma = -1 \Rightarrow \frac{1 + \sigma}{2} = 0, \quad \sigma = 1 \Rightarrow \frac{1 + \sigma}{2} = 1$$

- ▶ Exponent wird mit Bias gespeichert: $E + 1023$
- ▶ Mantisse speichert die Bits $z_1, \dots, z_{52}$

Spezielle Exponentenwerte

Die zwei freien Exponenten -1023 und 1024
(bzw. 0 und 2047 in Bias-Darstellung)
werden für Sonderfälle benutzt.

Verwendung:

- ▶ ± 0
- ▶ $\pm \infty$
- ▶ subnormale Zahlen (Exponent kleiner als -1022)
- ▶ NaN (*Not a Number*)

NaN entsteht z. B. bei unzulässigen Rechenoperationen wie

$$\log(-1).$$

Ebenso können Werte größer als 1023 über spezielle Kodierungen dargestellt werden.

Range eines Maschinenzahlbereichs

Wir bezeichnen mit

$$\text{range}(F) := [\min\{f \in F \mid f > 0\}, \max F]$$

das kleinste Intervall, in dem alle darstellbaren positiven Zahlen liegen.

4.3 Rundungen

Da es nur endlich viele Maschinenzahlen gibt, muss man sich mit gerundeten Darstellungen von reellen Zahlen zufrieden geben.

Definition 4.5: Rundung

Sei

$$F = F(b, m, E_{\min}, E_{\max})$$

ein Maschinenzahlbereich.

Eine Abbildung

$$\text{rd} : \mathbb{R} \rightarrow F$$

heißt **Rundung** (zu F), wenn für alle $x \in \mathbb{R}$ gilt:

$$|x - \text{rd}(x)| = \min_{a \in F} |x - a|.$$

Bedeutung:

- ▶ x : reelle Zahl
- ▶ $\text{rd}(x)$: gerundete Maschinenzahl
- ▶ gewählt wird die Zahl aus F , die am nächsten an x liegt

Mehrere mögliche Rundungen

Man beachte, dass es mehrere Rundungen gibt.

Zum Beispiel kann in

$$F(10, 2, 0, 2)$$

die Zahl

12,5

auf 12 oder 13 gerundet werden.

Sonderfall: x liegt genau zwischen zwei Zahlen.

Bei kaufmännischer Rechnung wird x im Zweifel zur betragsmäßig größeren Zahl gerundet.

IEEE 754: Runden zur geraden Zahl

IEEE 754 fordert:

x wird so gerundet, dass die letzte Stelle gerade wird.

Also zum Beispiel:

$$\text{rd}(12,5) = 12 \quad \text{und} \quad \text{rd}(13,5) = 14$$

Ein Grund dafür ist, dass bei Rechnungen wie

$$(\dots (((12 + 0,5) - 0,5) + 0,5) - \dots)$$

sonst abwechselnd auf- und abgerundet wird.

Bei kaufmännischer Rundung divergiert diese Reihe.

Vorteil der Regel:

Die Regel „zur geraden Zahl runden“ wurde gewählt, weil sich Rundungsfehler statistisch besser ausgleichen.

Definition 4.6: Absoluter und relativer Fehler

Definition 4.6. Es seien $x, \tilde{x} \in \mathbb{R}$, wobei $\tilde{x}$ eine Näherung von x sei.

Dann heißt

$$|x - \tilde{x}|$$

der **absolute Fehler**.

Falls $x \neq 0$, heißt

$$\left| \frac{x - \tilde{x}}{x} \right|$$

der **relative Fehler**.

- ▶ Absoluter Fehler = tatsächliche Differenz.
- ▶ Relativer Fehler = proportionale Abweichung bezogen auf x .

Beispiel: Absoluter und relativer Fehler

Beispiel:

$$x = 3,145 \quad \text{und} \quad \tilde{x} = 3,14$$

Absoluter Fehler:

$$|x - \tilde{x}| = |3,145 - 3,14| = 0,005$$

Relativer Fehler:

$$\left| \frac{x - \tilde{x}}{x} \right| = \frac{0,005}{3,145} \approx 0,0016$$

Als Prozentwert:

$$100 \cdot 0,0016 = 0,16\%$$

Das ist die prozentuale Abweichung von $\tilde{x}$ zu x .

Maschinengenauigkeit

Die **Maschinengenauigkeit** ist der maximale relative Fehler, der bei einer Rundung von Zahlen im darstellbaren Bereich auftreten kann.

Also:

größte relative Abweichung, die durch Rundung entstehen kann

Bedeutung:

schlimmste prozentuale Ungenauigkeit, die dein Zahlensystem noch verursacht

eps-Definition

Definition: Sei F ein Maschinenzahlbereich. Die Maschinengenauigkeit von F wird mit $\text{eps}(F)$ bezeichnet und ist definiert durch

$$\text{eps}(F) := \sup \left\{ \left| \frac{x - \text{rd}(x)}{x} \right| \mid x \in \mathbb{R}, |x| \in \text{range}(F), \text{rd Runding zu } F \right\}.$$

- ▶ sup = kleinste obere Schranke / größter möglicher Grenzwert.
- ▶ Das ist der größte relative Fehler, der auftreten kann.
- ▶ Genau dieser Wert beschreibt die Maschinengenauigkeit.

Satz 4.8

Satz 4.8

Für jeden Maschinenzahlbereich

$$F = F(b, m, E_{\min}, E_{\max}) \quad \text{mit} \quad E_{\min} < E_{\max}$$

gilt:

$$\text{eps}(F) = \frac{1}{1 + 2b^{m-1}}$$

Das hängt ab von:

- ▶ Basis b
- ▶ Mantissenlänge m

Je größer m , desto genauer das System.

Beispiel 4.9

Für den durch IEEE 754 definierten Typ `double` gilt:

$$\text{eps}(F_{\text{double}}) = \frac{1}{1 + 2^{53}} \approx 1,11 \cdot 10^{-16}$$

Das ist die Maschinengenauigkeit vom Typ `double`.

Hinweis zu `epsilon()` in C++

Man beachte, dass

`std::numeric_limits<double>::epsilon()`

die kleinste Zahl x liefert mit

$$x > 0 \quad \text{und} \quad 1 + x \in F_{\text{double}}.$$

Es gilt daher:

$$x = 2^{-52} \approx 2 \cdot \varepsilon(F_{\text{double}})$$

Wichtig:

- ▶ Das ist der Abstand von 1 zur nächsten größeren darstellbaren `double`-Zahl.
- ▶ Nicht exakt dieselbe Definition wie die theoretische Maschinengenauigkeit $\varepsilon(F)$.

Definition 4.10: Signifikante Stellen

Sei F ein Maschinenzahlbereich und $s \in \mathbb{N}$.

Eine Maschinenzahl $f \in F$ hat (*mindestens*) s *signifikante Stellen* in der b -adischen Gleitkommadarstellung, falls $f \neq 0$ und für jede Rundung rd sowie jede Zahl $x \in \mathbb{R}$ mit $rd(x) = f$ gilt:

$$|x - f| \leq \frac{1}{2} b^{\lfloor \log_b |f| \rfloor + 1 - s}$$

Bedeutung:

- ▶ Der Rundungsfehler darf höchstens so groß sein, dass s Stellen zuverlässig korrekt bleiben.
- ▶ Je größer s , desto genauer ist f .
- ▶ $\lfloor \cdot \rfloor$ bezeichnet die Gaußklammer (Abrunden auf ganze Zahl).

Definition 4.10: Signifikante Stellen

Sei F ein Maschinenzahlbereich und $s \in \mathbb{N}$.

Eine Maschinenzahl $f \in F$ hat (*mindestens*) s *signifikante Stellen* in der b -adischen Gleitkommaarstellung, falls $f \neq 0$ und für jede Rundung rd sowie jede Zahl $x \in \mathbb{R}$ mit $rd(x) = f$ gilt:

$$|x - f| \leq \frac{1}{2} b^{\lfloor \log_b |f| \rfloor + 1 - s}$$

Wie liest man den Exponenten?

$$\lfloor \log_b |f| \rfloor + 1 - s$$

- ▶ $\lfloor \log_b |f| \rfloor$: Größenordnung von f (wo die führende Ziffer liegt)
- ▶ $+1$: Verschiebung auf die erste signifikante Stelle
- ▶ $-s$: gehe s Stellen nach rechts
- ▶ Dadurch trifft man genau die Stelle, bis zu der die Zahl sicher korrekt ist.

Bedeutung:

Dies ist das mathematische Kriterium dafür, dass die gespeicherte Zahl f auf mindestens s signifikante Stellen genau ist.

Definition 4.10: Signifikante Stellen mit Beispiel

Eine Maschinenzahl $f \in F$ hat mindestens s signifikante Stellen, wenn

$$|x - f| \leq \frac{1}{2} b^{\lfloor \log_b |f| \rfloor + 1 - s}$$

gilt.

Beispiel: $f = 1234$, $s = 3$, $b = 10$

$$\lfloor \log_{10}(1234) \rfloor + 1 - s = 3 + 1 - 3 = 1$$

$$10^1 = 10 \quad \Rightarrow \quad \frac{1}{2} \cdot 10 = 5$$

Also darf der Fehler höchstens 5 sein:

$$|x - 1234| \leq 5$$

$$x \in [1229, 1239]$$

Interpretation:

Die ersten 3 signifikanten Stellen (1, 2, 3) bleiben stabil im praktischen Sinn.

Was sind signifikante Stellen?

- ▶ 1234,56 hat 6 signifikante Stellen.
- ▶ 0,00452 hat 3 signifikante Stellen (4, 5, 2).
- ▶ Führende Nullen zählen nicht.

Beispiel 4.11: Signifikante Stellen bei double

Jede Maschinenzahl $f \in F_{\text{double}} \setminus \{-\max F_{\text{double}}, 0, \max F_{\text{double}}\}$ hat mindestens

$$\lfloor 52 \cdot \log_{10}(2) \rfloor = 15$$

signifikante Stellen in der Dezimaldarstellung.

Erklärung:

- ▶ Double hat 52 Bits in der Mantisse (plus hidden bit).
- ▶ Umrechnung Binär $\rightarrow$ Dezimal:

$$\log_{10}(2) \approx 0,301$$

- ▶ Daher:

$$52 \cdot 0,301 \approx 15,65 \Rightarrow \lfloor \cdot \rfloor = 15$$

Bedeutung:

Ein double kann etwa 15–16 signifikante Dezimalstellen darstellen.

Beispiel 4.11: Fehlerabschätzung bei double

Für $x \in \mathbb{R}$ mit normalisierter Gleitkommadarstellung

$$x = \sigma \cdot 2^E \cdot \sum_{i=0}^{\infty} z_i 2^{-i} \quad \text{und} \quad rd(x) = f$$

gilt wegen $2^E \leq |f|$:

$$|x - f| \leq \frac{1}{2} \cdot 2^E \cdot 2^{-52} = 2^{-53} |f|$$

Umrechnung in Dezimaldarstellung:

$$2^{-53} = 10^{-53 \log_{10} 2} \approx 10^{-15,95}$$

$$\Rightarrow |x - f| \lesssim \frac{1}{2} \cdot 10^{\lfloor \log_{10} |f| \rfloor + 1 - 52 \log_{10} 2}$$

Schlussfolgerung:

Der Rundungsfehler eines double ist klein genug, um etwa 15 signifikante Dezimalstellen zu garantieren.

C++: Ausgabe mit setprecision

Nach

```
#include <iomanip>
```

kann man mit

```
std::cout << std::setprecision(15);
```

festlegen, dass bei nachfolgenden Ausgaben vom Typ double etwa 15 signifikante Stellen ausgegeben werden.

Beispiel:

```
#include <iostream>
```

```
#include <iomanip>
```

```
int main() {  
    double x = 1.0 / 3.0;  
  
    std::cout << std::setprecision(15)  
              << x << std::endl;  
}
```

Bedeutung:

- ▶ Nur Anzeigeformat wird geändert.
- ▶ Der gespeicherte Wert bleibt derselbe.
- ▶ Nützlich, um Genauigkeit sichtbar zu machen.

4.4 Maschinenzahlarithmetik

Sei $\circ$ eine der vier Operationen

$$\{+, -, \cdot, /\}.$$

Wie Beispiel 4.13 verdeutlicht, gilt für $x, y \in F$ nicht notwendigerweise:

$$x \circ y \in F.$$

Warum?

Weil das exakte mathematische Ergebnis oft nicht im endlichen Maschinenzahlssystem darstellbar ist.

Beispiel:

$$10^{300} \cdot 10^{300} = 10^{600}$$

Dies liegt außerhalb des erlaubten Exponentenbereichs.

$\Rightarrow$ Overflow / Underflow

Ersatzoperation auf dem Rechner

Stattdessen wird auf dem Rechner eine Ersatzoperation

$$\odot$$

ausgeführt, sodass

$$x \odot y \in F$$

gilt.

Wir nehmen im Folgenden an, dass:

Annahme 4.12

$$x \odot y = \text{rd}(x \circ y)$$

für eine Rundung rd zu F und alle $x, y \in F$.

Bedeutung:

- ▶ großes Kreiszeichen $\odot$: Rechneroperation
- ▶ kleines Kreiszeichen $\circ$: mathematische Operation $\{+, -, \cdot, /\}$
- ▶ wichtig, um Computerschritte mathematisch analysieren zu können

Hinweis zur Ausarbeitung

Dies wird zum Beispiel vom IEEE-Standard 754 gefordert.

Zudem wird dies dort auch für die Wurzelfunktion vorausgesetzt, nicht aber z. B. für

$$\sin(x), \quad \exp(x), \quad \text{etc.}$$

Zum Weiterlesen:

Die exakte Rundung elementarer Funktionen ist deutlich schwieriger als bei den vier Grundrechenarten.

Beispiel 4.13

Sei

$$F = F(10, 2, -5, 5).$$

Wähle

$$x = 4,5 \cdot 10^1 = 45 \quad \text{und} \quad y = 1,1 \cdot 10^0 = 1,1.$$

Dann sind:

$$x \oplus y = \text{rd}(x + y) = \text{rd}(46,1) = \text{rd}(4,61 \cdot 10^1) = 4,6 \cdot 10^1$$

$$x \ominus y = \text{rd}(x - y) = \text{rd}(43,9) = \text{rd}(4,39 \cdot 10^1) = 4,4 \cdot 10^1$$

$$x \odot y = \text{rd}(x \cdot y) = \text{rd}(49,5) = \text{rd}(4,95 \cdot 10^1) \in \{4,9 \cdot 10^1, 5,0 \cdot 10^1\}$$

$$x \oslash y = \text{rd}(x/y) = \text{rd}(40,\overline{90}) = \text{rd}(4,\overline{09} \cdot 10^1) = 4,1 \cdot 10^1$$

Relativer Fehler

Gemäß Annahme 4.12 gilt für den relativen Fehler:

$$\left| \frac{x \circ y - x \odot y}{x \circ y} \right| = \left| \frac{x \circ y - \text{rd}(x \circ y)}{x \circ y} \right| \leq \text{eps}(F)$$

falls

$$|x \circ y| \in \text{range}(F).$$

Dabei ist

$$x \odot y = \text{rd}(x \circ y).$$

Zur Erinnerung:

$$\text{eps}(F) := \sup \left\{ \left| \frac{x - \text{rd}(x)}{x} \right| \mid x \in \mathbb{R}, |x| \in \text{range}(F) \right\}.$$

Beispiel zum relativen Fehler

Beispiel:

$$x \circ y = 10,00 \quad (\text{exaktes Ergebnis})$$

$$x \odot y = 9,99 \quad (\text{gerundetes Ergebnis})$$

Absoluter Fehler:

$$0,01$$

Relativer Fehler:

$$\frac{0,01}{10} = 0,001 = 0,1\%$$

Wenn $\text{eps}(F) = 0,1\%$, dann passt die Aussage.

Merksatz:

Je kleiner $\text{eps}(F)$, desto genauer das Zahlensystem.

Wichtiger Zusatz

Das Kommutativgesetz für Addition und Multiplikation gilt auch in der Maschinenzahlarithmetik.

Assoziativ- und Distributivgesetz gelten jedoch im Allgemeinen nicht.

$$x \oplus y = y \oplus x$$

$$x \odot y = y \odot x$$

Aber im Allgemeinen nicht:

$$(x \oplus y) \oplus z = x \oplus (y \oplus z)$$

$$x \odot (y \oplus z) = (x \odot y) \oplus (x \odot z)$$

Beispiel 4.14

Im Maschinenzahlbereich

$$F = F(10, 2, -5, 5)$$

seien

$$x = 4,1 \cdot 10^0, \quad y = 8,2 \cdot 10^{-1}, \quad z = 1,4 \cdot 10^{-1}.$$

Dann sind:

$$(x \oplus y) \oplus z = 4,9 \oplus 0,14 = 5,0$$

$$x \oplus (y \oplus z) = 4,1 \oplus 0,96 = 5,1$$

$$x \odot (y \oplus z) = 4,1 \odot 0,96 = 3,9$$

$$(x \odot y) \oplus (x \odot z) = 3,4 \oplus 0,57 = 4,0$$

Warum ist das so?

Nach jedem Rechenschritt wird gerundet:

Rechnen $\rightarrow$ Runden $\rightarrow$ Weiterrechnen

Dadurch verändern sich Zwischenwerte.

Wichtige Erkenntnis

Folglich können mathematisch äquivalente Ausdrücke bei der Ausführung in der Maschinenzahlarithmetik zu wesentlich unterschiedlichen Ergebnissen führen, auch wenn der Input als Maschinenzahlen darstellbar ist.

Dies kann **dramatische Folgen** haben.

Übersetzung:

Auch wenn zwei Formeln mathematisch exakt dasselbe bedeuten, können sie auf dem Computer verschiedene Ergebnisse liefern.

Warum? Wegen:

- ▶ Rundung
- ▶ Reihenfolge der Rechenschritte
- ▶ Auslöschungseffekten
- ▶ Overflow / Underflow
- ▶ Verlust von Stellen

Folgen für die Praxis

Folgen für die Praxis:

- ▶ falsche Simulationen
- ▶ falsche wissenschaftliche Ergebnisse
- ▶ Abstürze
- ▶ ungenaue Finanzberechnungen
- ▶ Fehler in Technik / Navigation / Medizinsoftware

Wichtigste Erkenntnis des Kapitels:

In numerischer Mathematik zählt nicht nur, ob eine Formel mathematisch richtig ist,
sondern auch, **wie** man sie berechnet.

Quellenverzeichnis

- [1] Hougardy, Stefan und Vygen, Jens. *Algorithmische Mathematik*. 2. Auflage. Springer Spektrum, 2018.